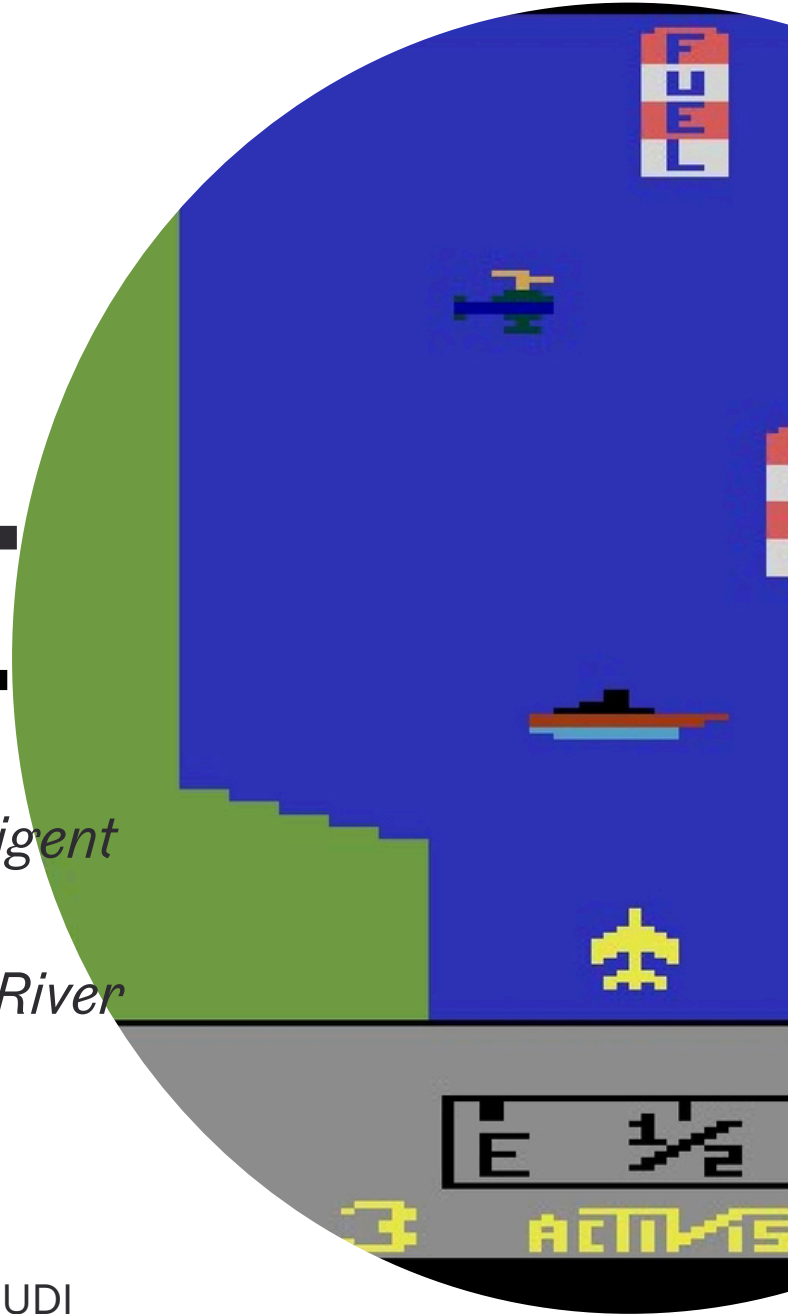




Université
de Limoges

RAPPORT DE PROJET

*Optimisation d'un Agent Intelligent
par Apprentissage par
Renforcement sur le jeu Atari River
Raid*



Yassir EL HAMOUDI



Du 10 avril au 28 mai 2026



Universite de limoges

Encadrant M. Tristan VACCON

Année universitaire : 2025-2026

REMERCIEMENTS

Avant d'aborder le développement de ce rapport, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué au bon déroulement de ce projet de fin d'études.

Je remercie chaleureusement mon encadrant de projet, **Monsieur Tristan VACCON**, pour sa disponibilité, sa patience, ainsi que pour la pertinence de ses conseils scientifiques tout au long de mes recherches sur l'apprentissage par renforcement. Son expertise m'a permis de surmonter les obstacles techniques liés à la manipulation des environnements d'émulation et des frameworks d'intelligence artificielle.

Mes remerciements s'adresse également à l'ensemble du corps enseignant de l'**Université de Limoges** pour la qualité de la formation qui m'a été dispensée, m'offrant les bases théoriques nécessaires à la concrétisation de ce travail.

Enfin, je tiens à remercier mes proches et mes collègues d'études, qui m'ont soutenu et encouragé durant toute la durée de ce projet.

SOMMAIRE

REMERCIEMENTS.....	1
PARTIE 1 : CONTEXTE ET FORMULATION DE LA PROBLÉMATIQUE.....	3
1.1 Présentation du jeu River Raid :	3
1.2 Présentation du projet et Problématique :	4
PARTIE 2 : ENVIRONNEMENT TECHNIQUE ET OUTILS UTILISÉS.....	4
2.1 L'interface de l'environnement : Gymnasium & ALE	4
2.2 Frameworks d'apprentissage : Stable-Baselines3 & PyTorch	5
2.3 Le traitement d'image et l'algorithme : CNN & PPO	5
2.3.1 La vision de l'IA : Le réseau de neurones convolutif (CNN)	5
2.3.2 Les Entrées et les Sorties du Réseau	6
2.3.3 Le fonctionnement du processus d'entraînement (L'algorithme PPO)	7
PARTIE 3 : PREMIÈRE EXPÉRIMENTATION - LE MODÈLE BRUT (BASELINE)	7
3.1 Configuration du script initial de test (Agent Aléatoire)	7
3.2 Implémentation du premier entraînement PPO (Modèle Brut)	8
3.3 Analyse du comportement observé et Limites de la Baseline :	9
PARTIE 4 : OPTIMISATION COMPORTEMENTALE - LE "REWARD ENGINEERING"	9
4.1 Implémentation du CustomRewardWrapper	9
4.2 Observations comportementales et problématique du carburant (FUEL)	10
PARTIE 5 : GESTION AVANCÉE DU CARBURANT (FUEL ENGINEERING).....	11
5.1 Phase d'observation et de diagnostic de la jauge	11
5.2 Implémentation du Wrapper de ravitaillement (Modèle v3)	12
CONCLUSION GÉNÉRALE.....	13
BIBLIOGRAPHIE	14

PARTIE 1 : CONTEXTE ET FORMULATION DE LA PROBLÉMATIQUE

1.1 Présentation du jeu River Raid :

Sorti en 1982 sur la console Atari 2600 et développé par Carol Shaw chez Activision, River Raid est un classique du jeu de tir à défilement vertical (shoot 'em up).

Dans ce jeu, le joueur pilote un avion de chasse jaune qui remonte un fleuve parsemé d'obstacles. Les mécaniques de jeu reposent sur plusieurs règles strictes :

- **Déplacement et Combat** : L'avion peut se déplacer de gauche à droite, accélérer, ralentir et tirer des missiles pour détruire les ennemis (bateaux, hélicoptères, puis avions de ligne dans les niveaux avancés).
- **Contraintes du terrain** : Le fleuve est délimité par des rives rocheuses (zones vertes). Tout contact avec les rives ou un appareil ennemi entraîne le crash immédiat de l'avion et la perte d'une vie.
- **Gestion du carburant (Le défi majeur)** : L'avion consomme continuellement du carburant. Sa jauge (située en bas de l'écran) diminue en permanence. Pour ravitailler, le joueur doit survoler des jerricans marqués "FUEL". Cependant, une subtilité technique s'impose : si le joueur tire sur un jerrican, celui-ci est détruit (rapportant quelques points), mais l'opportunité de ravitaillement est perdue, menant inéluctablement à la panne sèche.



Figure1 : Un exemple de partie : le joueur controle l'avion jaune

1.2 Présentation du projet et Problématique :

L'objectif central de ce projet est d'appliquer le Deep Reinforcement Learning (Apprentissage par Renforcement Profond) pour concevoir un agent autonome capable de maîtriser River Raid.

Contrairement à la programmation classique où l'on écrit des règles fixes ("si un ennemi approche, va à gauche"), l'approche par renforcement consiste à laisser une Intelligence Artificielle découvrir par elle-même les meilleures stratégies en jouant des millions de fois.

La Problématique du projet est la suivante :

En optimisant les hyperparamètres algorithmiques et en sculptant les signaux de feedback par l'ingénierie des récompenses (Reward Engineering), est-il possible d'amener une IA à piloter l'appareil en toute sécurité, à rationaliser sa consommation de carburant et à détruire le pont final pour valider le premier niveau du jeu ?

Ce travail nous confronte à un défi scientifique majeur en IA : le dilemme de la récompense à long terme. L'agent doit en effet apprendre à inhiber un gain immédiat (le score obtenu en détruisant un jerrican de carburant) au profit d'une stratégie de survie indispensable sur le long terme (le survol de ce même jerrican pour éviter la panne sèche).

PARTIE 2 : ENVIRONNEMENT TECHNIQUE ET OUTILS UTILISÉS

2.1 L'interface de l'environnement : Gymnasium & ALE

Pour que l'Intelligence Artificielle puisse interagir avec un jeu conçu dans les années 1980 comme River Raid, il est indispensable d'utiliser une interface de communication standardisée. Dans le cadre de ce projet, cette liaison est assurée par le couple Gymnasium et ALE :

- ALE (Arcade Learning Environment) / ale-py : Il s'agit du moteur d'émulation sous-jacent. Cet écosystème open-source émule la console Atari 2600 et permet à un script Python d'exécuter le jeu à une vitesse largement supérieure à celle d'une console réelle. En le configurant en mode headless (sans affichage graphique pour l'humain), les ressources de calcul sont optimisées pour accélérer l'entraînement.

- **Gymnasium** : Développée par la Farama Foundation pour succéder à la célèbre bibliothèque OpenAI Gym, cette plateforme fournit le cadre de travail (framework) standard pour l'Apprentissage par Renforcement. Elle structure le cycle de l'agent autour de trois variables fondamentales interconnectées :
 - L'Observation : La capture d'écran brute du jeu (le flux de pixels) reçue par l'IA à chaque étape.
 - L'Action : La commande envoyée par l'agent au joystick virtuel (Tirer, Gauche, Droite, Accélérer, Ralentir).
 - La Récompense : Le signal numérique renvoyé par l'environnement qui permet à l'IA d'évaluer l'efficacité de sa dernière action.

2.2 Frameworks d'apprentissage : Stable-Baselines3 & PyTorch

Une fois l'environnement de jeu modélisé par Gymnasium, il est nécessaire d'exploiter des outils capables de gérer les calculs lourds de l'Intelligence Artificielle. Pour cela, deux bibliothèques majeures ont été articulées :

- **Stable-Baselines3 (SB3)** : Il s'agit d'une bibliothèque de référence en Python qui fournit des implémentations hautement optimisées et fiables d'algorithmes d'Apprentissage par Renforcement. Dans le cadre de ce projet, nous utilisons l'algorithme PPO (Proximal Policy Optimization). SB3 permet de configurer cet algorithme, de gérer les hyperparamètres (comme le taux d'apprentissage ou le facteur de réduction) et de sauvegarder les modèles entraînés de manière structurée.
- **PyTorch** : C'est le moteur de calcul qui tourne en arrière-plan de Stable-Baselines3. PyTorch est une bibliothèque open-source d'apprentissage automatique (Machine Learning) spécialisée dans le calcul tensoriel. C'est elle qui prend en charge la création, la mise à jour et l'optimisation mathématique du réseau de neurones artificiels (le "cerveau" de notre agent) en exploitant la puissance de calcul de la carte graphique (GPU) ou du processeur (CPU).

2.3 Le traitement d'image et l'algorithme : CNN & PPO

Le traitement des données visuelles et la prise de décision automatisée reposent sur une architecture mathématique et structurelle précise qui transforme un flux de pixels bruts en commandes de jeu optimales :

2.3.1 La vision de l'IA : Le réseau de neurones convolutif (CNN)

Nous utilisons une politique de réseau de neurones convolutif (CnnPolicy) pour traiter les informations spatiales du jeu.

- **Les Données d'Entrée** : Pour réduire la charge de calcul, l'image brute de l'émulateur Atari est convertie en niveaux de gris et compressée en une dimension réduite de 84 par 84 pixels.
- **La Séquence Temporelle** : De plus, le wrapper VecFrameStack empile les 4 dernières images consécutives. L'entrée totale injectée dans le réseau est donc un tenseur de dimensions (4, 84, 84). Cette étape est cruciale : une seule image fixe ne permet pas de percevoir le mouvement. En observant 4 images d'un coup, le CNN peut calculer la vitesse, l'accélération et la trajectoire des éléments à l'écran.
- **Le Nombre de Couches** : Le tenseur passe ensuite à travers trois couches de convolution successives pour extraire les caractéristiques visuelles clés (les rives, les ennemis et les jerricans) :
 - La première couche applique 32 filtres de taille 8 par 8 avec un pas (stride) de 4.
 - La deuxième couche applique 64 filtres de taille 4 par 4 avec un pas de 2.
 - La troisième couche applique 64 filtres de taille 3 par 3 avec un pas de 1.
- **La Fonction d'Activation** : Après chaque couche convolutive, la fonction d'activation ReLU (Rectified Linear Unit) est systématiquement appliquée pour introduire de la non-linéarité dans le modèle. Le résultat est aplati (Flatten) et connecté à une couche cachée entièrement connectée (Dense Layer) de 512 neurones, elle aussi activée par une fonction ReLU.

2.3.2 Les Entrées et les Sorties du Réseau

Le modèle utilise une structure Acteur-Critique qui sépare le choix des actions de l'évaluation des états :

- **Entrée** : Le tenseur d'images empilées de dimensions (4, 84, 84) représentant l'état visuel actuel du jeu.
- **Sortie de l'Acteur (La Décision)** : Une couche linéaire dont la taille correspond exactement au nombre d'actions discrètes disponibles dans River Raid (tirer, gauche, droite, accélérer, ralentir). Une fonction d'activation Softmax convertit ces valeurs en probabilités de choix pour chaque commande.
- **Sortie du Critique (L'Évaluation)** : Une couche linéaire dotée d'une sortie unique (un scalaire sans fonction d'activation) qui estime la valeur mathématique attendue par l'agent à long terme depuis cet état.

2.3.3 Le fonctionnement du processus d'entraînement (L'algorithme PPO)

Développé par OpenAI, l'algorithme Proximal Policy Optimization (PPO) orchestre l'apprentissage de manière itérative selon un cycle précis :

- **Phase de Collecte (Rollout)** : L'agent interagit avec l'environnement Gymnasium pendant un bloc fixe de pas de temps ($n_steps = 128$). Durant cette phase, il stocke en mémoire les images observées, les actions choisies et les récompenses obtenues.
- **Calcul de l'Avantage** : À la fin de la collecte, l'algorithme calcule si les actions choisies par l'Acteur ont donné de meilleurs ou de pires résultats que l'estimation moyenne calculée par le Critique.
- **Optimisation par Mini-Batches** : Les données récoltées sont mélangées et divisées en petits groupes ($batch_size = 64$). Le réseau met alors à jour ses poids en effectuant une descente de gradient via le moteur PyTorch pour maximiser l'efficacité de sa politique.
- **Le Mécanisme de Clipping (Élagage)** : La grande force de PPO réside dans sa fonction de clipping (optimisation proximale, réglée par défaut à 0.2) : elle bride mathématiquement les changements de comportement trop radicaux d'une mise à jour à l'autre. Cela évite que l'IA n'oublie brutalement ce qu'elle a appris suite à un accident de vol, garantissant un entraînement stable et régulier sur le fleuve.

PARTIE 3 : PREMIÈRE EXPÉRIMENTATION - LE MODÈLE BRUT (BASELINE)

3.1 Configuration du script initial de test (Agent Aléatoire)

Avant de lancer l'entraînement intensif d'un réseau de neurones avec Stable-Baselines3, une phase préliminaire essentielle consiste à valider la communication entre Python, l'interface Gymnasium et l'émulateur Atari. Pour cela, un script de test de l'environnement a été développé afin de vérifier la boucle d'interaction de base (Action/Observation/Récompense).

Le script initial a été configuré selon la logique suivante :

- **Initialisation ET Création de l'environnement et rendu graphique** : Le moteur est enregistré via `gym.register_envs(ale_py)` et l'environnement est instancié avec `gym.make("ALE/Riverraid-v5", render_mode="human")`. Le paramètre `render_mode="human"` est activé ici pour ouvrir une fenêtre graphique à l'écran, permettant à un observateur humain de contrôler visuellement le bon déroulement de la simulation.
- **Politique d'action aléatoire (Random Baseline)** : Pour tester l'environnement sans modèle d'intelligence artificielle entraîné, À chaque étape (durant 1 000 cycles), l'agent choisit ses commandes (tirer, tourner, accélérer) totalement au hasard à l'aide de la méthode `env.action_space.sample()`.
- **Gestion du cycle de vie** : Le script surveille les variables `terminated` (mort) et `truncated` (fin du temps) à chaque pas de temps (`env.step`). Si l'une d'elles devient vraie, l'environnement est réinitialisé par `env.reset()`.

3.2 Implémentation du premier entraînement PPO (Modèle Brut)

Une fois l'environnement validé, nous avons exécuté un premier véritable algorithme d'apprentissage profond en utilisant la configuration PPO standard de Stable-Baselines3 couplée à la politique de traitement d'images `CnnPolicy`:

```

train_ppoV1.py > ...
1  import ale_py
2  import gymnasium as gym
3
4  from stable_baselines3 import PPO
5
6  print("Création environnement...")
7
8  env = gym.make(
9      "ALE/Riverraid-v5",
10     render_mode=None
11 )
12
13 print("Création modèle PPO...")
14
15 model = PPO(
16     "CnnPolicy",
17     env,
18     learning_rate=0.00025,
19     gamma=0.99,
20     n_steps=128,
21     batch_size=64,
22     ent_coef=0.01,
23     verbose=1)
24
25 print("Début apprentissage...")
26
27 model.learn(total_timesteps=50000)
28
29 print("Sauvegarde modèle...")
30
31 model.save("riverraid_ppo_v1")
32
33 env.close()
34
35 print("FIN")

```

Figure 2 : Script entraînement PPO (Modèle Brut)

3.3 Analyse du comportement observé et Limites de la Baseline :

L'exécution combinée de la politique purement aléatoire et de ce premier entraînement PPO brut met en évidence les limites de l'agent et pose les bases de notre problématique:

- **Au niveau du test aléatoire (3.1) :** L'avion effectue des mouvements saccadés et chaotiques, confirmant qu'un agent jouant au hasard est incapable de progresser. Il s'écrase contre les rives ou les ennemis en quelques secondes.
- **Au niveau du modèle PPO brut (3.2) :** Grâce au réseau convolutif, l'IA commence à stabiliser sa trajectoire sur le fleuve. Cependant, elle se retrouve immédiatement bloquée par le dilemme de la récompense à long terme. Ne recevant que le score brut d'Atari comme signal de retour, l'IA associe la destruction des jerricans de "FUEL" à un gain de points immédiat. Elle les détruit systématiquement au lieu de les survoler pour reconstituer sa jauge, provoquant sa perte par panne sèche après seulement quelques instants de vol.

PARTIE 4 : OPTIMISATION COMPORTEMENTALE - LE "REWARD ENGINEERING"

4.1 Implémentation du CustomRewardWrapper

Pour surmonter le dilemme du score brut identifié lors de l'analyse de la Baseline, les signaux de rétroaction transmis à l'agent ont été mathématiquement restructurés via l'ingénierie des récompenses (Reward Engineering). Cette démarche a été matérialisée par le développement d'un Wrapper personnalisé, baptisé CustomRewardWrapper, qui intercepte la fonction `step()` de l'environnement afin de modifier à la volée la récompense renvoyée à l'algorithme PPO avant la mise à jour des poids du réseau de neurones.

Le mécanisme algorithmique mis en place repose sur une modification dynamique du score d'origine d'Atari selon deux conditions strictes appliquées à chaque pas de temps :

- **Le bonus de survie (if not morte) :** Tant que l'avion reste en vie, une récompense cumulative et positive de +0.2 est ajoutée à chaque cycle pour inciter l'algorithme à stabiliser sa trajectoire et à prolonger la durée du vol.
- **La pénalité de mort (if morte) :** Dès que l'agent entre en collision avec une rive ou un ennemi, la variable morte passe à l'état vrai et une lourde amende mathématique de -20 est appliquée pour ancrer la préservation de l'appareil comme une priorité absolue.

Afin de valider cette logique, l'architecture logicielle a été configurée pour exécuter un entraînement intensif de 500 000 pas de temps à l'aide du script suivant :

```

train_ppoV2.py > ...
7
8 # =====
9 # CUSTOM REWARD WRAPPER
10 # =====
11 # Ce wrapper permet de modifier les rewards du jeu
12 # avant qu'elles soient envoyées à PPO.
13 #
14 # Objectif :
15 # - encourager la survie
16 # - pénaliser la mort
17 # =====
18
19 class CustomRewardWrapper(Wrapper):
20
21     # Initialisation du wrapper
22     def __init__(self, env):
23         super().__init__(env)
24
25     # Fonction appelée à chaque action PPO
26     def step(self, action):
27
28         # Exécute l'action dans le jeu
29         # et récupère les informations du jeu
30         obs, recompense, morte, tronque, info = self.env.step(action)
31
32         # BONUS SURVIE
33
34         if not morte :#si IA survie en donne une recompense
35             recompense += 0.2
36
37         # PÉNALITÉ MORT
38         if morte : #si IA et morte en donne une penalite
39             recompense -= 20
40
41         # Retourne les nouvelles données à PPO
42         return obs, recompense, morte, tronque, info
43

```

Figure3 : Script un Wrapper personnalisé, CustomRewardWrapper

4.2 Observations comportementales et problématique du carburant (FUEL)

L'analyse de l'agent entraîné avec ce Wrapper met en évidence deux comportements distincts :

- **Contrôle et esquiv** : Grâce à l'association du bonus (+0.2) et de la pénalité (\$-20), l'IA stabilise parfaitement sa trajectoire au centre du fleuve. Face aux obstacles et aux ennemis, elle déclenche des manœuvres d'évitement précises, ce qui valide l'apprentissage de la survie.
- **Problématique du carburant (FUEL)** : N'ayant aucune règle liée à l'essence codée dans ce Wrapper, l'agent ignore les jerricans de carburant ou continue de les détruire. L'avion finit donc systématiquement par s'écraser suite à une panne sèche lorsque la jauge descend à zéro.

Cette approche itérative a permis de sécuriser d'abord le vol et l'esquive, posant les bases nécessaires pour intégrer ensuite la gestion du carburant via la lecture de la mémoire RAM du jeu.

PARTIE 5 : GESTION AVANCÉE DU CARBURANT (FUEL ENGINEERING)

5.1 Phase d'observation et de diagnostic de la jauge

Avant de modifier l'algorithme pour intégrer le comportement de ravitaillement, un script de diagnostic dédié a été développé. Ce programme charge le modèle entraîné sur la survie (riverraid_ppo_v2), active le rendu graphique visuel (render_mode="human"), et intercepte en temps réel toutes les récompenses positives renvoyées par l'interface Gymnasium pour identifier à quel moment précis l'appareil interagit avec les éléments du jeu :

```
31
32 while True:
33
34     # PPO choisit une action
35     action, _states = model.predict(obs)
36
37     # Exécute action dans le jeu
38     obs, recompense, morte, tronque, info = env.step(action)
39
40     # =====
41     # AFFICHAGE REWARDS
42     # =====
43
44     if recompense > 0:
45         print("Reward détectée :", recompense)
46
47     # =====
48     # SI ÉPISODE TERMINÉ
49     # =====
50
51     if morte or tronque:
52
53         print("Épisode terminé")
54
55         obs, info = env.reset()
56
57     # =====
58     # FERMETURE
59     # =====
```

Figure4 : Script observation comportement avec carburantà(fuel)

La mise en exécution de ce script de test a permis de découvrir que l'émulateur Atari attribue une valeur de score brute spécifique d'une valeur exacte de \$30\$ lorsque l'agent entre en contact avec un élément de carburant.

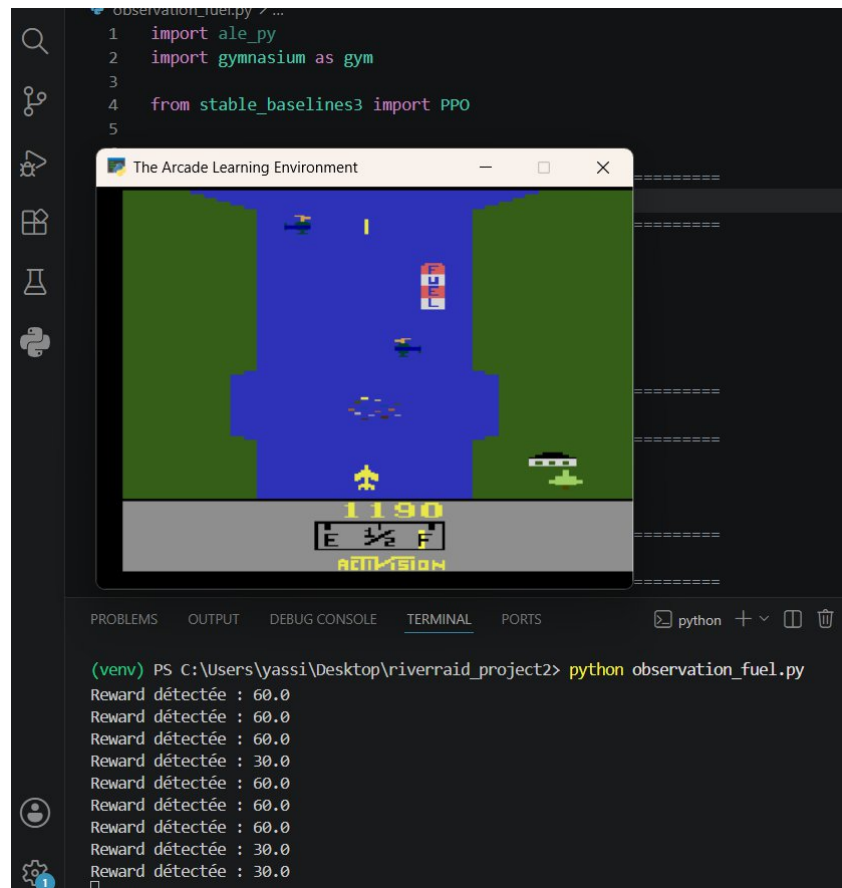


Figure5 : Interception de la récompense spécifique de 30 lors de l'interaction avec le carburant

5.2 Implémentation du Wrapper de ravitaillement (Modèle v3)

Pour résoudre le conflit qui poussait l'IA à détruire les jerricans, la modification technique majeure a été directement intégrée au niveau du Wrapper, sans altérer la structure native de l'algorithme PPO. C'est cette couche intermédiaire, CustomRewardWrapper, qui a été enrichie pour le modèle finalisé `riverraid_ppo_v3`.

Le Wrapper intercepte désormais en temps réel la récompense brute d'Atari. Dès qu'il détecte la valeur spécifique de 30 (contact avec le carburant), il y injecte à la volée un bonus de +3. Le script d'entraînement complet s'articule ainsi .

Grâce à cette modification localisée dans le Wrapper, la fonction de récompense globale transmise à PPO équilibre enfin les trois piliers indispensables à la réussite du jeu : l'esquive des rives (pénalité de -20), la constance de la trajectoire (bonus de +0.2)et la pérennité de l'énergie (bonus fuel de +3)

CONCLUSION GÉNÉRALE

La réalisation de ce projet dédié à l'optimisation d'un agent autonome sur le jeu Atari River Raid a permis de valider avec succès les concepts clés de l'apprentissage par renforcement profond. En articulant l'algorithme PPO avec une architecture de traitement d'images par réseau de neurones convolutif (CNN), nous avons pu observer la transition d'une politique de contrôle initialement chaotique vers un comportement hautement stratégique et fluide. Ce travail a été une expérience aussi passionnante que formatrice, rythmée par de vrais défis techniques et de longues phases d'incompréhension face aux choix de l'algorithme. Développer un agent autonome s'est avéré être un exercice bien moins linéaire que prévu, m'obligeant à passer d'une posture de simple codeur à celle d'un analyste rigoureux du comportement de l'IA.

Ce projet m'a confronté à la dure réalité du Deep Learning : la frustration de voir une Baseline tourner pendant des milliers de cycles pour finalement s'écraser bêtement en boucle, ou pire, détruire délibérément sa propre jauge de carburant pour gratter quelques points à court terme. C'est à travers ces échecs successifs que j'ai compris la portée concrète du Reward Engineering. En concevant le CustomRewardWrapper, j'ai dû apprendre à "penser" comme le réseau de neurones, à doser chaque micro-bonus de survie (+0.2) et chaque pénalité de crash (-20) pour guider l'algorithme PPO sans le bloquer dans des minimums locaux.

La plus grande satisfaction de ce travail reste sans doute la résolution empirique du problème du carburant pour le modèle `riverraid_ppo_v3`. Après avoir passé des heures à observer la console et à décortiquer les signaux bruts de l'émulateur lors des phases de test, la découverte de la récompense exacte de 30 points a été le déclic..

Au-delà des compétences techniques pures acquises en manipulant Gymnasium, Stable-Baselines3 et PyTorch, ce projet m'a énormément appris sur la démarche scientifique. J'ai pris conscience qu'en matière d'intelligence artificielle, l'humain ne doit pas simplement fournir de la puissance de calcul brute, mais qu'il doit jouer le rôle de guide en sculptant intelligemment l'environnement.

Pour la suite, le projet ouvre des perspectives stimulantes que j'aimerais explorer, notamment le fait d'allonger l'entraînement sur plusieurs millions de pas pour voir comment l'agent se comporte dans les niveaux supérieurs, ou encore le défi d'aller lire directement les octets dans la mémoire RAM de l'Atari pour offrir à l'IA une gestion encore plus fine et préventive de son essence.

BIBLIOGRAPHIE

[1] STABLE-BASELINES3 DEVELOPERS. Stable-Baselines3: Reliable Reinforcement Learning Implementations. Rapport technique officiel, Journal of Machine Learning Research, 2021.

Utilité technique : Cette référence sert de guide d'implémentation pour le script d'entraînement Python. Elle atteste de la fiabilité et de la conformité des structures de code algorithmiques (telles que PPO et la politique CnnPolicy) exploitées dans le cadre du projet.

[2] FARAMA FOUNDATION. Gymnasium: A Standard Interface for Reinforcement Learning Environments. Documentation officielle de référence, 2023.

Utilité technique : Ce document de référence sert à formaliser l'architecture de la boucle d'interaction agent-environnement. Il encadre la récupération standardisée des états d'observation, l'envoi des commandes discrètes au joystick virtuel, et la manipulation des variables de récompenses numériques.

[3] PYTORCH / META AI RESEARCH. PyTorch: An Imperative Style, High-Performance Deep Learning Library. Rapport d'architecture officiel, Advances in Neural Information Processing Systems, 2019.

Utilité technique : Cet article de référence sert à légitimer le choix du framework de calcul tensoriel sous-jacent. Il documente la méthode de calcul lourd des gradients et la gestion de l'optimisation des poids du réseau de neurones artificiels via la puissance de la machine.

[4] OPENAI RESEARCH. Proximal Policy Optimization Algorithms. Rapport technique officiel, arXiv, 2017.

Utilité théorique : Ce rapport d'architecture officiel sert à expliciter les fondements mathématiques de l'algorithme PPO sélectionné pour ce travail. Il décrit le fonctionnement de la fonction d'objectif avec clipping, indispensable pour stabiliser l'apprentissage et encadrer la mise à jour de la politique de l'avion sur le fleuve.

tion des fonctions d'activation nécessaires au traitement du flux d'images.

[5] TOWARDS DATA SCIENCE. Convolutional Neural Network: A Complete Guide. Guide technique officiel, 2018. [En ligne]. Disponible sur : [Convolutional Neural Network: A Complete Guide](#)

Utilité théorique : Ce guide technique exhaustif sert de support didactique pour documenter l'architecture interne des réseaux convolutifs. Il détaille de manière accessible le fonctionnement mathématique des filtres, des pas de déplacement (strides) et l'application des fonctions d'activation nécessaires au traitement du flux d'images.

lien Git : <https://github.com/ELhamoudi-yassir/Agent-Intelligent-par-Apprentissage-par-Renforcement-sur-le-jeu-A-River-Raid>